VM FRAMEWORK

Design Language Specification

Internal Agent Reference

Version 1.1 | March 2026 | Language-Agnostic Design. Rust Implementation.

Marko Tahvanainen

Purpose and scope

This document defines the design language used by all agents in the VM Framework engineering organisation when designing, specifying, and reviewing system components.

It is language-agnostic. Design decisions are made at the conceptual level first. Implementation notes for Rust are provided where the language has specific idioms that must be respected, but the design itself does not depend on them.

Every agent that produces or consumes design artifacts must follow this specification. The Software Designer produces designs using this language. The SPMs direct coders using it. The Code Reviewers enforce it. The Quality Manager measures against it.

> **Note:** This document is a living standard. When an agent encounters a design situation not covered here, it escalates to the Software Designer rather than inventing a convention. The SD updates this document and notifies all roles.

01 Naming conventions

Consistent names across the entire codebase. No dialects.

Names are the primary communication interface between components and between agents. Inconsistent naming is technical debt that compounds silently. These conventions are mandatory, not advisory.

General principles

- Names describe what something IS or what something DOES. Never how it does it.
- Names are unambiguous when read without context. If a name only makes sense inside the module that contains it, it is wrong.
- Abbreviations are prohibited except for the project-wide approved list below. Unknown abbreviations cost the next reader time they did not budget.
- Boolean names always read as a true/false question. is_active, has_error, can_restore — never active, error, restore for booleans.

Approved abbreviations

Abbreviation	Meaning
vm	Virtual machine
cow	Copy-on-write
cfg	Configuration
buf	Buffer
ctx	Context
err	Error

msg	Message
req	Request
res	Result (operation outcome)
resp	Response (reply to a request)
src	Source
dst	Destination
id	Identifier
idx	Index
len	Length
max	Maximum
min	Minimum

Naming patterns by construct

Construct	Pattern
Module	lowercase_snake. Single noun or noun phrase. vm_manager, cow_engine, storage_pool
Function / procedure	verb_noun or verb_noun_qualifier. create_snapshot, restore_vm_state, fetch_storage_pool
Data structure / type	PascalCase noun. VmConfig, SnapshotHandle, StoragePool
Interface / trait	PascalCase noun describing capability. Restorable, Configurable, Observable
Constant	UPPER_SNAKE_CASE. MAX_SNAPSHOT_DEPTH, DEFAULT_MEMORY_MB
Variable	lowercase_snake. snapshot_id, restore_duration_ms
Error type	PascalCase ending in Error. RestoreError, ConfigParseError, NetworkError
Test function	test_ prefix then what it verifies. test_restore_completes_within_timeout

> **Rule:** No name longer than 40 characters. If it takes 40 characters to name something, the thing being named is too broad.

02 Function and procedure design

One function. One job. One reason to change.

Functions are the atomic unit of design. Every design decision about how the system does things is expressed as a function contract. The contract is what matters. The implementation satisfies the contract.

Function contract

Every function must be fully specified before implementation begins. A function contract contains:

- Name — following naming conventions in Section 01
- Purpose — one sentence. What this function does. Not how.
- Inputs — each parameter named, typed, and described. Units included for numeric values.
- Output — the return value named, typed, and described. Including all possible error conditions.
- Preconditions — what must be true before calling this function. If preconditions are violated at runtime, the function returns a typed error. In debug builds, precondition violations trigger a debug_assert! and panic. Do not describe precondition violations as undefined behaviour — that term has a specific meaning in Rust relating to memory safety and must not be used loosely.
- Postconditions — what is guaranteed to be true after this function returns successfully.
- Side effects — any state changes outside the return value. If none, state explicitly.
- Error conditions — every failure mode listed explicitly with the error type produced.

Design rules

- A function does exactly one thing. If describing what it does requires the word 'and', it is two functions.
- A function is testable in isolation. If testing it requires standing up half the system, it is too large or too coupled.
- Functions that mutate state are separated from functions that return values. Do not mutate and return in the same function unless the mutation IS the meaningful return.
- Maximum parameter count: five. If a function needs more than five parameters, introduce a configuration structure.
- Recursive functions must document their termination condition explicitly. Unbounded recursion is a design defect.

Procedure vs function distinction

A function computes and returns a value. It has no side effects. Given the same inputs it always produces the same output.

A procedure performs an action and may change state. It may return a result indicating success or failure but the primary purpose is the action, not the value.

This distinction must be clear from the name and the contract. Mixed function-procedures that both compute a value and change state are a design smell and require explicit justification.

Note: Rust implementation note: Functions map to fn returning a value type. Procedures map to fn returning Result<(), ErrorType> or fn returning (). The distinction is enforced by the type system — use it.

03 Data structure design

Shape your data. The logic follows.

Data structures define the shape of the system's knowledge. A well-designed data structure makes invalid states impossible to represent. A poorly designed one allows them everywhere and pushes the burden of validation into every function that touches it.

Design rules

- Make illegal states unrepresentable. If a field combination is invalid, the structure should not allow it to exist. Use separate types for separate states.
- Every field has a purpose. No fields carried forward from earlier designs because removing them seemed risky. Unused fields are removed.
- Ownership is explicit. Every piece of data has one owner. Shared references are documented as such.
- Mutable and immutable data are separated. Configuration that is set once lives in a different structure from state that changes during operation.
- Nested structures have a maximum depth of three levels before the design should be reconsidered. Deep nesting hides complexity.

Required documentation per structure

- Purpose — what real-world concept or system state does this structure represent
- Invariants — conditions that must always hold for this structure to be valid
- Lifecycle — when is this structure created, modified, and destroyed
- Ownership — who owns this structure and who may hold references to it

Naming and organisation

- Related structures are grouped in the same module
- Structures that cross subsystem boundaries are defined in a shared types module, not in either subsystem
- Builder pattern is used for structures with more than five fields or with optional fields

Note: Rust implementation note: Prefer enum over bool fields for state. A VM that is either Running,

> Paused, Saved, or Error is safer than a VM with is_running, is_paused, is_saved booleans that can be in contradictory combinations.

04 Memory handling and ownership

Explicit ownership. No surprises. No leaks.

Memory design is ownership design. Every byte of memory has one owner. The design document must make ownership explicit before a line of code is written. Memory problems found at runtime are design problems that should have been found on paper.

Ownership model

- Single ownership: one owner per value at any point in time. The owner is responsible for the value's lifetime.
- Borrowed references: temporary access granted to another component. The borrower cannot outlive the owner.
- Shared ownership: used only when single ownership is genuinely impossible. Must be documented with justification.
- Transfer of ownership: when ownership passes between components, the design document must describe the transfer point explicitly.

Lifetime design rules

- No component holds a reference to data it does not own beyond the scope of a single operation unless the design document explicitly defines the lifetime relationship.
- Circular references are a design defect. If the design requires circular ownership, the ownership model is wrong.
- Long-lived allocations are named and tracked. Any allocation that lives beyond a single function call must be documented in the owning structure.

Resource management

- Every resource that is acquired must have a documented release path. File handles, network connections, VM handles, storage locks — all of them.
- Resources are released in the reverse order they were acquired.
- Error paths release resources acquired before the error. Partial acquisitions do not leak.
- Timeouts are designed in from the start for any resource acquisition that depends on external state. Waiting forever is not a design.

Buffer management

- Buffer sizes are defined as named constants, never as magic numbers.
- Buffer overflows are impossible by design, not prevented by runtime checks. The design must make the maximum input size knowable before allocation.
- Zero-copy operations are preferred for large data. If data must cross a subsystem boundary and is larger than 64KB, the design must justify the copy and document why zero-copy was not achievable. This threshold is a named constant SUBSYSTEM_COPY_THRESHOLD_BYTES defined in the shared constants module.

> **Warning:** Rust implementation note: Rust's ownership system enforces most of these rules at compile time. If you are fighting the borrow checker, the design is wrong, not the language. Redesign before reaching for unsafe.

05 Error handling design

Errors are part of the design. Not afterthoughts.

Every function that can fail must declare how it fails and what the caller is expected to do about it. An undocumented failure mode is a guarantee that someone will be surprised by it in production.

Error classification

Error class	Definition and handling
Recoverable	The operation failed but the system is still in a valid state. The caller decides whether to retry, fallback, or propagate. Returned as a typed error value.
Unrecoverable	The component is in an invalid state that cannot be corrected at runtime. Two scopes apply. Component-level unrecoverable: the specific operation fails and the component signals a fatal error to its supervisor; other components continue operating. Process-level unrecoverable: the entire process is in an invalid state — memory corruption, invariant violation that spans the whole system. Only process-level unrecoverable errors trigger controlled shutdown. Do not shut down the entire process because one VM restore failed.
Transient	The operation failed due to a temporary condition. Retry with backoff is appropriate. Must document maximum retry count and backoff policy.
User error	The caller provided invalid input. Returned as a typed validation error. Never panic. Always describe what was wrong and what is valid.

Error design rules

- Every error type carries enough context to diagnose the problem without additional logging. An error that says 'operation failed' is not an error type, it is a shrug.
- Error types are specific to their domain. A storage error and a network error are different types even if both result in operation failure.
- Errors are not used for flow control. If a condition is expected, handle it as a normal case, not an error.
- Error propagation paths are designed explicitly. When a low-level error propagates upward, each layer adds context without losing the original cause.
- Silent failure is forbidden. Every error must be either handled at the point of occurrence or explicitly propagated upward. Dropping an error without handling it requires documented justification.

Error context requirements

Every error value must contain:

- What operation failed
- Why it failed — the specific condition
- Where it failed — module and function
- The original cause if this error wraps a lower-level error

> **Note:** Rust implementation note: Use Result<T, E> for all recoverable errors. Use the ? operator to propagate. Use a project-wide error enum with variants per domain. Do not use unwrap() or expect() in production code paths — these are design defects, not style preferences.

---

06 Interface and API design

Contracts between components. Explicit, stable, versioned.

An interface is a contract between a provider and a consumer. The provider guarantees the contract. The consumer depends on it. Every interface must be designed as if the consumer and provider will be maintained by different teams who do not communicate informally.

Interface design rules

- Interfaces are minimal. They expose only what the consumer needs, not everything the provider has. Hidden internals cannot become accidental dependencies.
- Interfaces are stable. Once published to a consumer, an interface does not change without a version increment and a migration path.
- Interfaces are documented completely. Every function in the interface has a full contract as defined in Section 02.

- Interfaces hide implementation. The consumer must not be able to observe how the provider implements the contract. If implementation details leak through the interface, the interface is wrong.
- Input validation happens at the interface boundary. The provider validates all inputs before any internal processing. Garbage in, typed error out.

API versioning

- Every interface has a version number starting at v1.
- Additive changes — new functions added to the interface — are allowed within a version. New optional functionality is delivered through new functions or through configuration structs with optional fields, never through additional parameters on existing functions.
- Breaking changes — removed functions, changed function signatures, changed semantics, added required parameters — require a new version. In Rust, adding any parameter to an existing function breaks all callers. Treat all signature changes as breaking.
- Old versions are supported for a defined deprecation period documented at the time of the new version release.

Subsystem interface contracts

Each subsystem publishes exactly one interface to the outside world. Internal structures, helper functions, and implementation details are not part of the interface. Anything not in the published interface is private.

Interface contracts between subsystems are defined by the Software Designer and are the authoritative source of truth for both the provider and the consumer. Neither subsystem may deviate from the contract without a formal spec update.

External interface contracts

Functions that form the external user-facing API follow all interface design rules plus:

- All inputs sanitised before processing
- All outputs sanitised before returning — no internal paths, no internal identifiers, no internal error messages
- Authentication and authorisation checked before any operation executes
- Rate limiting applied at the interface layer, not inside the implementation

> **Note:** Rust implementation note: Use traits to define interfaces. Concrete types implement traits. Consumers depend on the trait, not the concrete type. This enforces interface hiding at the type system level.

07 Concurrency and threading design

Explicit parallelism. Designed-in safety. No data races by surprise.

Concurrency is not added to a design. It is designed in from the start or it is not used. Retrofitting concurrency onto a sequential design produces races, deadlocks, and behaviour that cannot be reasoned about. Every concurrent component is designed as such from the first draft.

Concurrency model

- The concurrency model for each subsystem is declared explicitly: single-threaded, multi-threaded with shared state, message-passing, or async/event-driven. The choice is documented with rationale.
- Shared mutable state between threads is a design smell. If two threads need to share mutable state, the design must justify it and document the synchronisation mechanism.
- Message passing is preferred over shared state for communication between concurrent components. Messages transfer ownership — the sending component relinquishes ownership on send. The receiving component has exclusive ownership. Messages do not need to be immutable; the ownership transfer provides the safety guarantee.
- Thread pools are sized explicitly. A thread pool of unbounded size is a resource exhaustion waiting to happen.

Synchronisation design rules

- Every lock is named and its purpose is documented. Anonymous locks are prohibited.
- Lock acquisition order is documented globally to prevent deadlocks. If lock A is ever acquired while holding lock B, that order must be consistent everywhere.
- Lock scope is minimised. The lock is held for the minimum time necessary. No I/O, no blocking calls, no function calls of unknown duration while holding a lock.
- Locks are never acquired inside error handling paths unless explicitly designed and documented.

Async design rules

- Async functions are async all the way up or not at all. Blocking inside an async context is a design defect.
- Timeouts are mandatory on all async operations that depend on external state. The timeout value is a named constant with documented rationale.
- Cancellation paths are designed explicitly. Every async operation that can be cancelled documents what state is left behind when cancellation occurs.

Warning: Rust implementation note: Rust prevents data races at compile time through the Send and Sync traits. If the compiler rejects a concurrent design, the design has a safety problem. Do not use unsafe to bypass this. Redesign.

08 External library declaration and fetching

Every dependency is a liability. Declare it. Justify it. Control it.

An external library is a dependency on code you did not write and cannot fully control. Every external dependency must be declared explicitly, justified, and managed. Undeclared dependencies are security risks and build reliability risks.

Dependency declaration requirements

Before any external library is introduced into the project, the SPM responsible for the subsystem submits a dependency declaration containing:

- Library name and current version
- Source repository and official URL
- Purpose — what capability this library provides that is not worth building
- Alternatives considered — at least two alternatives evaluated with reasons for rejection
- Licence — must be compatible with the project licence. Incompatible licences are rejected.
- Maintenance status — last release date, active maintainer, open security issues
- Security audit status — known CVEs checked at time of declaration
- Transitive dependencies — the full dependency tree documented

Rust: declaring dependencies in Cargo.toml

In a Cargo workspace, shared dependencies are declared under [workspace.dependencies] in the root Cargo.toml. Individual crates reference them with workspace = true. This ensures consistent versions across the entire project without repeating version strings.

```
[workspace.dependencies]
library-name = { version = "1.2.3", features = ["feature-name"] }
```

In an individual crate that belongs to the workspace:

```
[dependencies]
library-name = { workspace = true }
```

Version bounds in Cargo.toml should be semver-compatible ranges, not exact pins. Cargo.lock is committed to the repository and is the mechanism that guarantees reproducible builds. Exact pinning with = in Cargo.toml prevents even patch-level security updates from being applied without a manual edit.

```
# Correct — semver-compatible range, Cargo.lock provides reproducibility
tokio = { version = "1.35", features = ["rt", "net", "time"] }
```

```
# Wrong — exact pin prevents security patch updates
tokio = { version = "=1.35.1", features = ["rt", "net", "time"] }
```

Rust: fetching from repositories

The primary source is crates.io. For crates not available on crates.io or where a specific fork or commit is required:

```
# From a git repository at a specific commit
library-name = { git = "https://github.com/org/repo", rev = "abc1234" }

# From a local path (development only, never in a release build)
library-name = { path = "../local-library" }
```

Feature flags

Enable only the features required. Enabling all features pulls in transitive dependencies that are not needed and expands the attack surface.

```
# Correct — minimal features
serde = { version = "=1.0.195", features = ["derive"] }

# Wrong — all features
serde = { version = "=1.0.195", features = ["derive", "rc", "alloc",
"std"] }
```

Dependency review process

- All new dependencies require PPM awareness and Security role approval before introduction. The SPM submits the dependency declaration to both. Security checks CVE databases and licence compatibility. PPM approves from a scope and schedule perspective.
- cargo audit is run as part of the CI pipeline on every build. A build that introduces a dependency with a known CVE fails. This is automated. The Security role sets the policy; the pipeline enforces it.
- Security role reviews all new dependencies against known CVE databases before approval
- Dependency versions are reviewed at the start of each major release cycle
- Unused dependencies are removed. Cargo.toml is not a graveyard.

> **Warning:** A dependency that has not been updated in two years, has open security issues, or has no active maintainer is a risk. Document the risk assessment and approval explicitly before accepting it.

09 Module and subsystem boundary design

What you hide is as important as what you expose.

A module boundary is a promise. What is inside the boundary is the module's private concern. What is outside the boundary is the module's public contract. Blurring these boundaries creates coupling that makes the system impossible to change safely.

Boundary rules

- Every module has a single defined public interface. Only what is in the interface is accessible from outside the module.
- Modules do not reach into each other's internals. If module A needs something from module B's internals, the right answer is either to add it to B's interface or to reconsider the module boundary.
- Circular dependencies between modules are a design defect. If A depends on B and B depends on A, the boundary is wrong.
- The dependency direction is designed explicitly and documented. Dependencies flow in one direction only within a subsystem. The dependency graph is a DAG, not a mesh.

Subsystem boundary rules

- Subsystems communicate only through their published interface contracts as defined by the Software Designer.
- Shared data structures that cross subsystem boundaries live in a shared types module that neither subsystem owns exclusively.
- A subsystem's internal state is never directly readable or writable by another subsystem. All cross-subsystem state access goes through the interface.
- The Software Designer is the arbiter of all subsystem interface contracts. Neither subsystem may unilaterally change the contract.

> **Note:** Rust implementation note: Use the pub keyword deliberately. Everything is private by default. Only what is genuinely part of the public interface gets pub. pub(crate) for things that must be accessible within the crate but not outside it. pub(super) for things that must be accessible to the parent module only.

10 State management design

Where state lives. Who owns it. How it moves.

State is the hardest part of any system to reason about. Uncontrolled state is the source of most runtime bugs that are hard to reproduce. The design must make the location, ownership, and mutation paths of every significant piece of state explicit.

State classification

State class	Definition
Configuration state	Set at startup or by explicit user action. Does not change during normal operation. Immutable after initialisation.
Operational state	Changes during normal operation in response to system events. Owned by a single component. Access by others is through the owning component's interface.
Transient state	Exists for the duration of a single operation. Created on entry, destroyed on exit. Never escapes the operation scope.
Shared state	Accessible by multiple components. Requires explicit synchronisation design. Used only when operational state cannot satisfy the requirement.
Persistent state	Survives process restart. Stored to durable storage. Has a defined schema and migration path.

State design rules

- Every significant piece of state is classified using the table above. Unclassified state is a design gap.
- State mutation paths are explicit. For each piece of operational state, the design documents which functions may mutate it and under what conditions.
- State transitions are designed as a finite state machine where the state has discrete valid values. Undefined transitions are impossible by design.

> **Note:** Rust implementation note: The type-state pattern is the idiomatic way to enforce FSM correctness at compile time. Each state is a distinct type. Transition functions consume the current state type and return the next state type. An invalid transition cannot be called because the type does not exist. The VM state machine in Section 10 should be implemented this way.

- State that must survive failure is identified at design time and its persistence mechanism is specified before implementation.

> **Note:** The VM state machine is an example: a VM is in exactly one of — Created, Starting, Running, Pausing, Paused, Saving, Saved, Restoring, Stopping, Stopped, Error. Transitions between states are defined explicitly. An invalid transition is a typed error, not undefined behaviour.

11 Logging and observability design

Designed-in visibility. Not added later when something breaks.

Observability is a design requirement, not an operational concern. A system that cannot be observed cannot be diagnosed. Every significant operation, state transition, and error condition must be observable without modifying the production system.

Log levels

Level	When to use
ERROR	A condition that requires human attention. The operation failed in a way that may indicate a system problem. Logs at this level are actionable.
WARN	A condition that is unusual but does not require immediate action. A recoverable error, a retry, a threshold approaching.
INFO	Normal significant events. VM started, snapshot created, restore completed. Sparse. An INFO log storm is a design problem.
DEBUG	Information useful during development and diagnosis. Not enabled in production by default. Must not contain sensitive data.
TRACE	Fine-grained internal execution flow. Development only. Never in production.

Logging design rules

- Every log message includes: timestamp, log level, module, function, and a human-readable message.
- Log output is structured. Fields beyond the message body — VM ID, snapshot ID, operation ID, duration — are emitted as key-value pairs, not embedded in the message string. Structured logs can be queried. Embedded strings cannot.
- Log messages describe what happened in past tense. 'Snapshot created' not 'Creating snapshot'.
- Identifiers are included in log messages for correlation. VM ID, snapshot ID, operation ID — whatever allows log lines to be traced to a specific operation.
- Sensitive data is never logged at any level. VM contents, user credentials, private keys — never in logs.
- Log volume at INFO level must be manageable under normal load. If INFO logs are too frequent to read, the level choice is wrong.

Metrics design

- Every significant operation has a duration metric. Restore time, snapshot time, VM start time.
- Error rates are measured per operation type, not in aggregate.
- Resource usage metrics — memory, CPU, storage — are emitted at a defined interval.
- Metric names follow the naming conventions in Section 01.

12 Testing design

Designed to be tested. Not tested after being designed.

Testability is a design property. A component that is hard to test has a design problem. The test design is produced alongside the component design, not after implementation.

Test levels

Level	What it tests
Unit test	A single function or method in isolation. All dependencies are replaced with controlled substitutes. Tests the contract defined in Section 02.
Integration test	A component interacting with its real dependencies within a controlled environment. Tests that the interface contracts from Section 06 are honoured.
System test	The full system against defined scenarios. Tests that the functional specification is met end to end.
Property test	Invariants that must hold across a range of inputs. Generates inputs automatically. Tests the guarantees stated in data structure invariants.

Testability design rules

- Every function that has external dependencies — I/O, time, randomness, external services — accepts those dependencies as parameters. This makes them replaceable in tests.
- Test fixtures are designed alongside the component. The data needed to test a component is part of the component's design.
- Test coverage targets are defined at design time, not measured after the fact. The target for each module is documented in the design.
- Tests are deterministic. A test that sometimes passes and sometimes fails is a broken test. Flaky tests are fixed before they are merged.
- Test names describe what they test and what outcome is expected. test_restore_vm_returns_error_when_snapshot_not_found is a test name. test_restore is not.

> **Note:** The restore mechanism has a five-second performance requirement. This is a testable requirement. There is a test that measures restore duration and fails if it exceeds five seconds. Performance requirements without performance tests are wishes, not requirements.

13 Code documentation conventions

Every function. Every structure. Every interface. Documented before implementation.

Documentation is part of the design artifact, not an addition to the implementation. A component whose design cannot be expressed in documentation has not been fully designed.

Documentation requirements by artifact

Artifact	Required documentation
Module	Purpose, scope, what it owns, what it depends on, key design decisions made and rejected.
Public function	Full contract as defined in Section 02. All fields populated. No partial contracts.
Private function	Purpose and any non-obvious behaviour. Does not require full contract if used only within a single module.
Data structure	Purpose, invariants, lifecycle, ownership as defined in Section 03.
Public interface	Full contract for every function in the interface plus the overall interface purpose and versioning information.
Error type	What condition this error represents and what the caller is expected to do about it.
Constant	What the value represents, what it controls, and why this specific value was chosen.

Documentation rules

- Documentation is written before implementation. A function without documentation has not been designed. It has been improvised.
- Documentation describes the WHAT and the WHY. The implementation describes the HOW. These are different things. Do not document the implementation in the design documentation.
- Examples are included for non-obvious usage. A correct example is worth ten paragraphs of prose.
- Documentation is updated when the design changes. Stale documentation is worse than no documentation. It actively misleads.

> **Rule:** No public function, structure, or interface may be submitted for code review without complete documentation. The Code Reviewer rejects undocumented public artifacts without further review.

14 Design change process

Designs change. The process for changing them does not.

A design document is a baseline, not a prophecy. Implementations reveal things that designs did not anticipate. The process for handling design changes is as important as the original design process.

Change triggers

- Implementation reveals that the design is incorrect or incomplete
- A spec correction request from Quality or Security identifies a design-level problem (see Org Structure v2.0, Fix 3)
- An interface dispute between SPMs requires binding resolution (see Org Structure v2.0, Fix 4)
- Post-release feedback identifies a design-level correction (see Org Structure v2.0, Fix 6)

Change process

- The change request is submitted to the Software Designer with the specific design element that needs to change, the reason it needs to change, and the proposed change.
- The SD assesses the impact on dependent components and interfaces.
- If the change affects an interface contract, the SD produces an updated spec and notifies all affected SPMs.
- The SD updates this document if the change affects a design convention.
- The SD produces a reflection document for human review if the change affects the fundamental architecture.
- No implementation changes are made against a design that is under active change. The change is resolved first.

Version control of design documents

- This document is versioned. The version number increments on every substantive change.
- The change log records what changed, who requested it, and why.
- Previous versions are retained. A design decision that was reversed can be understood in context.

> **Note:** Design documents live in the repository alongside the code they describe. They are code reviewed the same way code is reviewed. A design document that has not been reviewed is not a baseline.